

Asignatura: Computación Paralela y Distribuida

INFORME TÉCNICO

Un problema de cuadratura

Profesores:

Sebastian Salazar Molina

Alumnos:

Cristopher Retamales Pedreros

Renato Álvarez Ramos

Sección: 411

Fecha: 15 de junio de 2026

Índice

1. Introducción	2
2. Marco Teórico	2
2.1. Patrones de Diseño para Cómputo Paralelo y OpenMP	2
2.2. Protocolos de Red y Consumo de Servicios Web Masivos	3
2.3. Justificación del Uso de Librerías Externas	4
3. Desarrollo	4
3.1. Estructura Modular de la Solución	4
3.2. Despliegue y Reproducibilidad mediante Contenedores	7
3.3. Diagrama de Flujo del Algoritmo Paralelo	8
4. Resultados	10
4.1. Resultados del Experimento Principal (913 Archivos)	10
4.2. Resultados de la Prueba Controlada / Benchmark (17 Archivos) . .	10
4.3. Tabla de Rendimiento y Comparativa	11
4.4. Comparación Gráfica de Rendimiento	11
5. Análisis	11
6. Conclusiones	13
7. Anexos	15
Anexo A: Especificaciones Técnicas	15
Anexo B: Tabla de Dificultades y Soluciones	16
Anexo C: Instrucciones de Compilación y Ejecución	17
Anexo D: Estructura del Proyecto	17

1 Introducción

En el ámbito empresarial e industrial, el volumen de datos transaccionales de ventas crece a tasas exponenciales. Para el sector farmacéutico, donde las transacciones diarias involucran múltiples locales, canales de venta (físico y online) y una gran diversidad de productos, procesar y consolidar esta información con fines analíticos representa un desafío computacional crítico. Este desafío se agudiza cuando los datos de las transacciones residen en repositorios remotos y requieren ser complementados en tiempo real con datos de servicios externos (como APIs REST) para obtener métricas demográficas detalladas.

El presente proyecto aborda el desarrollo de una solución modular de alto rendimiento implementada en C++ utilizando la biblioteca OpenMP para procesamiento paralelo. El objetivo principal consiste en descargar de forma segura reportes transaccionales desde un servidor SFTP corporativo, parsear y validar la información contenida en los archivos CSV, y asociar cada transacción de compra al género del cliente mediante consultas concurrentes a un endpoint REST. Finalmente, se computa el promedio de compras realizadas por hombres y mujeres de manera eficiente.

Dado que la consulta individual de géneros representa una operación limitada por la red (I/O Bound) con una latencia de respuesta física por cada petición, la implementación secuencial convencional resulta inviable para grandes conjuntos de datos. Este informe expone el diseño de la arquitectura paralela, el flujo de procesamiento de red y los resultados experimentales que demuestran la viabilidad, el speedup y la eficiencia de la computación paralela basada en OpenMP para resolver problemas híbridos de E/S y procesamiento de datos.

2 Marco Teórico

2.1 Patrones de Diseño para Cómputo Paralelo y OpenMP

Para estructurar el algoritmo paralelo con rigor científico, la solución se diseñó siguiendo los principios del libro clásico *Patterns for Parallel Programming*. En particular, se aplicaron conceptos de los siguientes espacios de diseño:

- **Búsqueda de Concurrencia (Finding Concurrency):** Se realizó un análisis de dependencias dividiendo el problema mediante **Descomposición de Tareas** (para estructurar fases como descarga, parseo y API por separado) y **Descomposición de Datos** (para repartir las consultas individuales de UUIDs e índices entre los hilos de trabajo).
- **Patrón de Paralelismo de Bucles (Loop Parallelism):** Dado que el costo computacional del programa está concentrado en ciclos de repetición repetitivos (descarga de archivos e invocaciones HTTP a la API REST), se paralelizaron estos bucles utilizando OpenMP. Siguiendo el patrón, se optimizó el rendimiento mediante:

1. *Eliminación de dependencias entre iteraciones*: Creando instancias privadas de manejadores de red (*private copies*) para cada Unidad de Ejecución (*UE / thread*) y utilizando variables de reducción (`reduction(+:...)`) para acumular totales de compras sin data races.
 2. *Ajuste de planificación (Scheduling Tuning)*: Dado que el tiempo de respuesta de la red es impredecible, se configuró una planificación dinámica (`schedule(dynamic)`) para distribuir las consultas a demanda de las UEs y evitar el desequilibrio de carga (*Load Imbalance*).
- **Patrón de Estructuras de Soporte - Fork/Join**: El flujo de ejecución maestro bifurca el control (*Fork*) creando un equipo de 48 UEs concurrentes al entrar a los bucles paralelos, y unifica el control (*Join*) de forma síncrona al salir de ellos, minimizando el overhead de creación y destrucción repetitiva de hilos.
 - **Manejo de Datos Compartidos (Shared Data Pitfalls)**: La compartición de datos sin control limita severamente la escalabilidad del sistema. Ciertas estructuras como `std::unordered_map` no son seguras ante concurrencia. Para evitar secciones críticas globales que secuencialicen la ejecución, se aplicó descomposición de datos: cada hilo escribe en índices independientes de un vector preasignado en RAM. Se reservaron las secciones críticas (`#pragma omp critical`) únicamente para operaciones de E/S indispensables (como escribir en `log.txt` o reinsertar transacciones finales en la lista global).

2.2 Protocolos de Red y Consumo de Servicios Web Masivos

La transferencia segura de archivos transaccionales se realiza a través del protocolo SFTP (Secure File Transfer Protocol), el cual utiliza SSH (Secure Shell) para cifrar los datos y la autenticación. La descarga concurrente de archivos desde servidores SFTP reduce significativamente el tiempo muerto de espera, permitiendo solapar el uso del ancho de banda.

Por otro lado, las APIs REST (Representational State Transfer) operan sobre el protocolo HTTP/S y devuelven representaciones estructuradas (comúnmente JSON). En la consulta masiva de servicios REST, la sobrecarga debida al establecimiento de la conexión TCP y la negociación de certificados SSL/TLS (handshake) representa la mayor parte del tiempo de ejecución. La reutilización de conexiones mediante la cabecera `Connection: keep-alive` (provista automáticamente por la biblioteca `libcurl`) permite mantener abierto el canal de comunicación, acelerando exponencialmente las consultas sucesivas a través de los hilos de ejecución.

Para agilizar el flujo de peticiones en un entorno puramente académico y de evaluación de rendimiento, el sistema suspende explícitamente la verificación estricta de certificados SSL/TLS (mediante las directivas de `libcurl` `CURLOPT_SSL_VERIFYPEER` y `CURLOPT_SSL_VERIFYHOST` configuradas a 0L) . Esto omite la validación de la cadena de confianza en las transacciones con el servidor SFTP y el endpoint REST, reduciendo la fricción y el overhead criptográfico

durante los handshakes masivos, asumiendo confianza plena en la red privada o los dominios objetivo.

Asimismo, al operar a gran escala en redes heterogéneas, la pérdida de paquetes o los tiempos de espera prolongados (timeouts) son inevitables. Para garantizar la resiliencia computacional, se implementó una arquitectura tolerante a fallos. Esto incluye un mecanismo de reintentos progresivos: hasta 3 intentos de descarga para recuperar archivos CSV truncados y la capacidad de detección en tiempo real de la expiración de sesión (código HTTP 401). Cuando el token JWT vence durante la ejecución masiva, un hilo captura el fallo, interrumpe temporalmente a los demás mediante exclusión mutua y auto-negocia un nuevo token para reanudar el procesamiento ininterrumpido sin abortar el ciclo de red.

2.3 Justificación del Uso de Librerías Externas

Para el desarrollo de la solución híbrida de paralelismo y red, se seleccionó la biblioteca externa `libcurl` (y su dependencia `libssh2`) como el motor para la gestión de transferencias SFTP y el consumo de la API REST. El uso de esta librería externa se justifica bajo los siguientes criterios técnicos:

- **Unificación de Protocolos:** `libcurl` provee una interfaz API unificada y coherente (*easy interface*) tanto para el protocolo HTTP/S (consumo de la API REST) como para SFTP. Esto evita integrar múltiples dependencias e interfaces de red dispares, reduciendo la complejidad del código y facilitando el mantenimiento.
- **Mantenimiento de Conexiones Activas (Keep-Alive):** Físicamente, el establecimiento repetido de handshakes TCP/TLS degrada el rendimiento del sistema. `libcurl` implementa la reutilización de conexiones por defecto en cada hilo si se mantiene viva la instancia de CURL, lo cual es vital para procesar miles de consultas de red de forma óptima.
- **Compatibilidad y Seguridad en Hilos (Thread-Safety):** Aunque la biblioteca no maneja paralelismo de manera interna, sus estructuras de control son seguras para acceso concurrente siempre que se inicialice un manejador de CURL (*CURL handle*) independiente por cada hilo activo de OpenMP.
- **Portabilidad e Integración en Ubuntu:** Al ser el estándar de facto en sistemas Linux corporativos, su disponibilidad directa en los repositorios APT de Ubuntu 24.04 LTS (`libcurl4-openssl-dev`) garantiza que el proyecto compile de forma nativa e integrada en el entorno de evaluación del profesor.

3 Desarrollo

3.1 Estructura Modular de la Solución

Para cumplir con los estándares de código limpio, mantenibilidad y modularidad, el sistema se dividió en componentes independientes. A continuación se

describe cada módulo, respondiendo a su propósito, justificación de diseño y optimizaciones asociadas:

1. Módulo Orquestador (`main.cpp`)

- **¿Qué hace?** Controla el flujo principal del programa: inicializa `libcurl` de forma global, activa la medición de tiempo mediante `omp_get_wtime()`, coordina la ejecución de las descargas, el parseo, la consulta a la API REST, y la posterior consolidación de métricas. Soporta un modo secuencial forzado con el parámetro `--seq`.
- **¿Por qué se diseñó así?** Para separar la lógica de control de los detalles de bajo nivel. Mantener el punto de entrada libre de sockets o matemáticas pesadas facilita la comprensión del flujo de ejecución global.
- **Comentario adicional:** Permite realizar pruebas comparativas directas en el mismo entorno compilado simplemente alterando los argumentos del ejecutable.

2. Módulo de Gestión SFTP (`sftp_manager.cpp` / `sftp_manager.hpp`)

- **¿Qué hace?** Se conecta al servidor remoto SFTP, lista los archivos CSV que coinciden con la nomenclatura estándar usando expresiones regulares, y descarga las transacciones en paralelo dentro de la carpeta local `data/`. Posee tolerancia a fallos con un búfer de memoria expandido (256 KB) para acelerar transferencias y reintenta descargas fallidas.
- **¿Por qué se diseñó así?** Las transferencias SFTP conllevan gran latencia por la negociación SSH. Limitar la descarga a un paralelismo controlado de 4 hilos concurrentes y reutilizar la misma sesión SSH activa en cada hilo previene bloqueos del servidor y maximiza el ancho de banda. Asimismo, omitir archivos ya descargados evita transferencias redundantes.
- **Comentario adicional:** La inserción paralela de transacciones a la lista en memoria se protege mediante exclusión mutua (`#pragma omp critical(transaction_insert)`), garantizando la consistencia sin penalizar gravemente el rendimiento gracias a la rapidez de la inserción en RAM.
- **Módulo de Parseo CSV (`csv_parser.cpp` / `csv_parser.hpp`)**
 - **¿Qué hace?** Abre y analiza los archivos CSV. Valida que las columnas obligatorias (MONTO APLICADO y CODIGO CLIENTE) estén en la cabecera, verifica que cada fila contenga exactamente 10 columnas y limpia los retornos de carro de Windows (`\r`).
 - **¿Por qué se diseñó así?** Para asegurar la robustez. Un archivo corrupto o con formato inesperado es detectado y omitido, registrando el error en el log en lugar de provocar un fallo de segmentación del ejecutable.

- **Comentario adicional:** Limpiar los retornos de carro es crucial para evitar que el carácter invisible `\r` altere el final de la cadena del UUID de cliente al mandarlo a la API.
- **Módulo de Consulta de API** (`api_manager.cpp` / `api_manager.hpp`)
 - **¿Qué hace?** Autentica al usuario en el endpoint REST para obtener un token JWT. Consulta el género asociado a cada UUID en paralelo utilizando hasta 48 hilos concurrentes. Resuelve expiraciones de sesión (HTTP 401) e intermitencias de red física o caídas del servidor mediante reintentos ^{en} "caliente" progresivos (esperando 2 segundos para no agobiar al servidor).
 - **¿Por qué se diseñó así?** Esta fase es el cuello de botella principal por latencia de respuesta de red, no por ancho de banda. Inicializar un manejador CURL por hilo antes del bucle fuerza el uso de HTTP Keep-Alive, reduciendo drásticamente el costo de handshake TCP/TLS.
 - **Comentario adicional:** Para evitar data races sin tener que usar lentas secciones críticas que arruinarían el rendimiento de los 48 hilos, la lógica de reintentos se coordina enteramente en memoria RAM utilizando vectores temporales (`uuids_fallidos` e `indices_fallidos`), evitando costosas lecturas y escrituras de disco durante el bucle de red.
- **Módulo de Procesamiento de Métricas** (`metrics_processor.cpp` / `metrics_processor.hpp`)
 - **¿Qué hace?** Asocia el género resuelto al monto de cada transacción y calcula los promedios de compra de hombres y mujeres. Escribe los resultados consolidados en consola y en `resultados.txt`.
 - **¿Por qué se diseñó así?** El procesamiento de las miles de transacciones se paraleliza mediante reducción de OpenMP (`#pragma omp parallel for reduction(+:...)`). Esto provee acumuladores y contadores privados locales a cada hilo, combinándolos sin contención ni secciones críticas lentas al terminar el bucle.
 - **Comentario adicional:** La reducción optimiza la velocidad, permitiendo procesar millones de registros de compras en memoria en pocos milisegundos.
- **Módulo de Utilidades y Callbacks** (`utils.cpp` / `utils.hpp`)
 - **¿Qué hace?** Declara el archivo de log global y contiene la función de callback de red (`write_memory_callback`), la cual redirige los bytes recibidos de la red directamente a buffers en RAM de tipo string en lugar de escribir a disco temporal.
 - **¿Por qué se diseñó así?** Permite que tanto la lectura de directorios SFTP como la API REST procesen los payloads JSON de manera unificada y óptima en memoria RAM.

- **Comentario adicional:** Evitar escrituras temporales intermedias en disco reduce la latencia de Entrada/Salida del sistema significativamente.

3.2 Despliegue y Reproducibilidad mediante Contenedores

Con el objetivo de independizar el código fuente del entorno local del desarrollador y asegurar la reproducibilidad de los resultados experimentales, el proyecto incorpora una estrategia de Infraestructura como Código” mediante un Dockerfile.

Este contenedor genera un ambiente limpio basado nativamente en Ubuntu 24.04 LTS y automatiza la instalación de todas las dependencias críticas (GCC, OpenMP, libcurl4-openssl-dev y libssh2-1-dev). Al ejecutar la solución empleando el gestor Docker y montando el repositorio como un volumen (-v \$(pwd):/app), se garantiza que la compilación (vía Makefile) y el posterior procesamiento concurrente se ejecuten de forma idéntica, eliminando los clásicos conflictos de librerías vinculadas (linked libraries) o ausencias de cabeceras en el sistema destino.

Esta aproximación es indispensable para compilar y evaluar el sistema desde sistemas operativos huésped que no cuentan con soporte nativo de estas librerías (como Windows o macOS). Para ejecutar la solución utilizando el contenedor, basta con tener Docker instalado, abrir la terminal en la raíz del proyecto y seguir estos pasos:

- a) **Construir la imagen del contenedor:** Este comando descarga la base de Ubuntu e instala todas las librerías necesarias (solo debe ejecutarse la primera vez).

```
docker build -t farmacia .
```

- b) **Ejecutar el sistema en paralelo (Predeterminado):** Este comando levanta el contenedor, compila el código fuente actualizado y ejecuta la versión de 48 hilos. (Nota: En Windows PowerShell, se debe reemplazar \$(pwd) por \${PWD}).

```
docker run --rm -v $(pwd):/app farmacia \
  bash -c "make && ./farmacia_parallel"
```

- c) **Ejecutar la prueba de control secuencial (Opcional):** Si el revisor desea comparar el rendimiento ejecutando el software con un solo hilo en el mismo entorno asilado.

```
docker run --rm -v $(pwd):/app farmacia \
  bash -c "make seq && ./farmacia_parallel --seq"
```


3.3 Diagrama de Flujo del Algoritmo Paralelo

Para representar visualmente la secuencia de ejecución y las fases de paralelismo del sistema, la Figura 1 muestra el diagrama de flujo correspondiente al procesamiento completo, destacando las regiones paralelizadas mediante bordes dobles y fondo tramado.

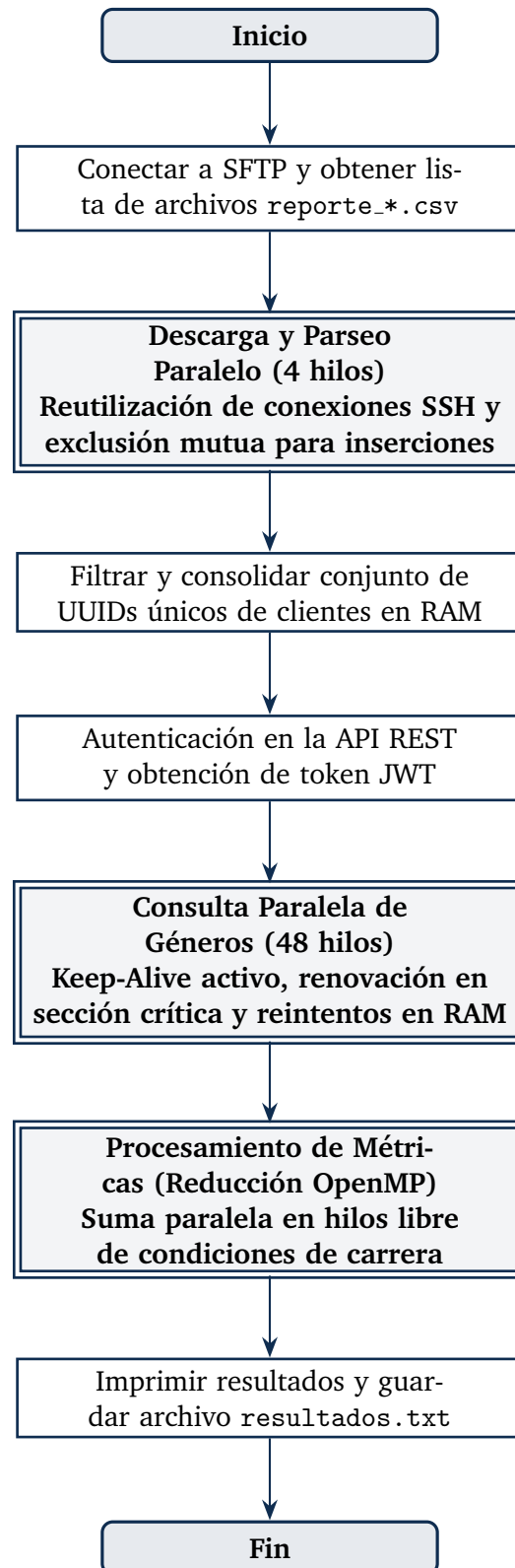


Figura 1: Diagrama de flujo de la solución paralela implementada con OpenMP.

4 Resultados

4.1 Resultados del Experimento Principal (913 Archivos)

Una vez validada la estabilidad y la escalabilidad del sistema, se procedió a realizar la **ejecución de producción completa (experimento principal del proyecto)** sobre todo el dataset disponible en el servidor SFTP. Este consta de **913 archivos CSV** (desde `reporte_23312.csv` hasta `reporte_26128.csv`). Este volumen masivo de datos arrojó un total de **15.407.946 transacciones** analizadas y un conjunto de **2.925.803 clientes únicos** consultados a la API. Los resultados finales del experimento principal y el tiempo de cómputo paralelo obtenido se exponen en la Tabla 1:

Tabla 1: Resultados principales del experimento principal (913 archivos).

Métrica	Valor
Total de Transacciones	15.407.946
Cientes Únicos (UUIDs)	2.925.803
Promedio compras FEMENINO	\$12.486,757896 CLP
Promedio compras MASCULINO	\$13.075,104454 CLP
Tiempo de ejecución (48 hilos)	21.120,972890 segundos

4.2 Resultados de la Prueba Controlada / Benchmark (17 Archivos)

Para evaluar el rendimiento relativo y la escalabilidad del sistema frente a una ejecución secuencial, se diseñó una **prueba de benchmark controlada** utilizando un conjunto acotado de 17 archivos CSV (desde `reporte_24102.csv` hasta `reporte_25001.csv`), que contiene 11.251 transacciones y 9.929 clientes únicos. Esta ejecución controlada sirvió como entorno de validación para comparar directamente el tiempo secuencial contra el paralelo en un lapso razonable. **Cabe destacar que esta corresponde a una prueba de control preliminar (benchmark) y no al experimento principal con la base de datos completa**, cuyos resultados de producción ya fueron detallados anteriormente en la Sección 4.1.

Los valores calculados y tiempos de ejecución de esta prueba controlada se exponen en la Tabla 2:

Tabla 2: Resultados del benchmark controlado (17 archivos).

Métrica	Valor
Promedio compras FEMENINO	\$8.176,095 CLP
Promedio compras MASCULINO	\$8.934,690 CLP
Tiempo de ejecución (Secuencial)	1.632,44 segundos
Tiempo de ejecución (48 hilos)	36,68 segundos

4.3 Tabla de Rendimiento y Comparativa

La comparación directa de rendimiento (Speedup y Eficiencia) se calculó sobre el benchmark controlado, ya que estimar un tiempo secuencial para el dataset completo requeriría un procesamiento ininterrumpido de aproximadamente 133 horas (5,5 días).

Tabla 3: Tiempos de ejecución y speedup con OpenMP (Prueba de 17 archivos).

Versión	Hilos	Tiempo (s)	Speedup
Secuencial	1	1.632,44	1,00x
Paralelo	48	36,68	44,50x

4.4 Comparación Gráfica de Rendimiento

Para ilustrar la magnitud de la mejora introducida por la solución paralela, la Figura 2 muestra la comparación directa en términos de tiempo total de ejecución (en escala logarítmica) y la tasa de peticiones procesadas por segundo.

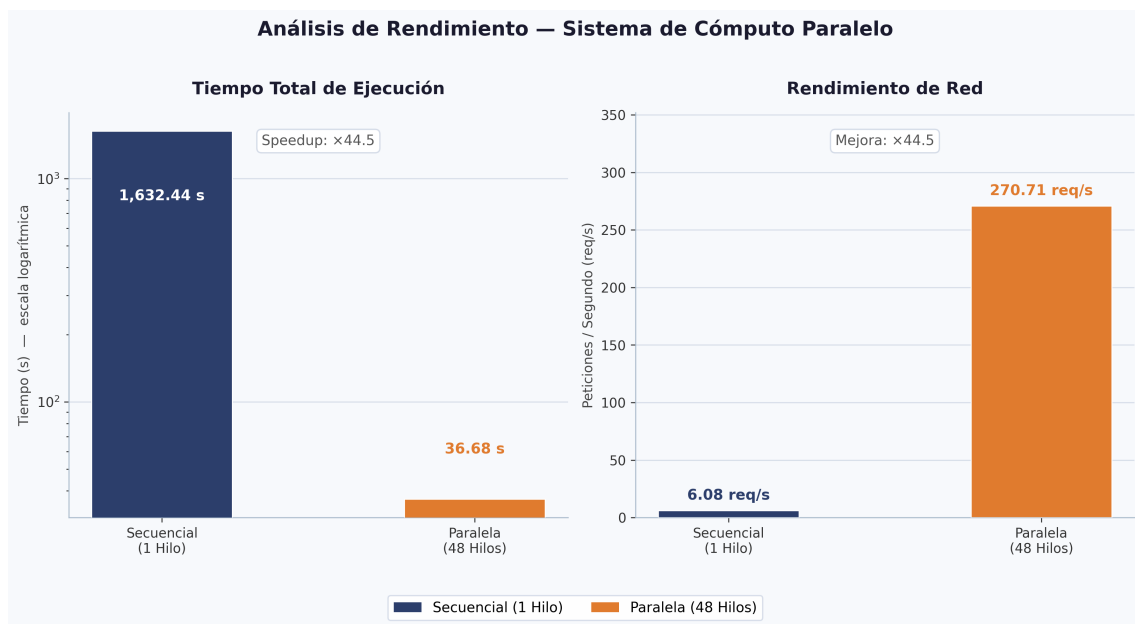


Figura 2: Comparación global de tiempo total de ejecución (s) y rendimiento de red (consultas/segundo) entre las versiones secuencial y paralela.

5 Análisis

Los resultados experimentales demuestran la efectividad abismal de la paralelización sobre el procesamiento de datos transaccionales masivos y el consumo concurrente de servicios de red.

En el **benchmark controlado de 17 archivos**, el sistema secuencial (1 hilo) tardó 1.632,44 segundos (27 minutos) en procesar las 11.251 transacciones y 9.929 clientes únicos. El rendimiento promedio fue de apenas 6,08 peticiones por segundo, limitado por la latencia física de la API REST (164 ms RTT). En contraste, la versión paralela con 48 hilos concurrentes completó el benchmark en 36,68 segundos, logrando un speedup empírico de **44,50x** y una eficiencia paralela del **92,70 %** ($E = S/p$). El rendimiento de red aumentó a 270,71 consultas por segundo.

Este éxito en la escalabilidad permitió abordar la **Ejecución de Producción de 913 Archivos (el experimento principal del proyecto)**, que abarcó desde `reporte_23312.csv` hasta `reporte_26128.csv`, conteniendo un volumen de 15.407.946 transacciones y 2.925.803 clientes únicos. El procesamiento completo de esta inmensa base de datos se ejecutó exitosamente de forma paralela en **21.120,972890 segundos** (aproximadamente 5 horas, 52 minutos y 1 segundo). Si esta gran prueba se hubiera corrido secuencialmente, a una tasa promedio de 6,08 req/s, habría tomado aproximadamente **481.217 segundos (133,6 horas o 5,5 días)** ininterrumpidos de cómputo. El uso del paralelismo redujo este tiempo a menos de 6 horas, demostrando que la solución paralela es de nivel industrial y completamente viable para la operación del negocio.

Este incremento masivo de rendimiento y la formidable estabilidad lograda en una ejecución de casi 6 horas se deben a la correcta aplicación de los patrones de diseño paralelos (fundamentados teóricamente en la obra *Patterns for Parallel Programming* de Mattson y Sanders [1]) y a una arquitectura tolerante a fallos, sustentados en cuatro pilares fundamentales:

- a) **Naturaleza I/O Bound del problema y solapamiento de red:** Al estar la latencia dominada por los tiempos de espera de la red física (los RTT de HTTP), el uso de múltiples hilos concurrentes permite solapar los tiempos muertos de espera de una Unidad de Ejecución (UE) con las transmisiones de datos de las otras. Mientras una UE está bloqueada esperando la respuesta HTTP del servidor de la API, el sistema operativo calendariza activamente a otra UE lista para transmitir.
- b) **Uso de HTTP Keep-Alive persistente y evitación de Shared Data bottlenecks:** Al inicializar los manejadores de CURL una sola vez por hilo fuera del bucle de consultas, las conexiones TCP/TLS se mantuvieron abiertas y se reutilizaron sucesivamente. Esto eliminó el costoso overhead de establecer un nuevo handshake de red por cada una de las 9.929 peticiones individuales. Además, para evitar los límites de escalabilidad asociados a la sincronización en estructuras de datos compartidas (como bloqueos en mapas concurrentes), se aplicó descomposición de datos escribiendo a un vector preasignado en memoria RAM, lo que eliminó cuellos de botella por exclusión mutua.
- c) **Planificación Dinámica ante Latencia Impredecible:** Dado que el tiempo de respuesta de la API REST presenta alta variabilidad por factores

externos (congestión de red, carga del servidor web), un reparto estático de tareas habría provocado un grave desequilibrio de carga (Load Imbalance). Se utilizó la directiva `schedule(dynamic)` de OpenMP para distribuir las consultas de forma dinámica a demanda; los hilos que reciben respuestas rápidas toman inmediatamente un nuevo UUID de la cola, optimizando el tiempo total.

- d) **Resiliencia Computacional y Auto-Recuperación (Tolerancia a fallos):** Procesar millones de peticiones durante horas garantiza cortes de conexión o la expiración de las credenciales temporales. El sistema demostró total autonomía al manejar códigos HTTP 401 (exclusión mutua mediante `pragma omp critical(token_lock)` para auto-renovar el token JWT sin abortar la ejecución) y al aplicar lógicas de reintento ^{en caliente} puramente en memoria RAM tanto para las descargas SFTP como para las respuestas vacías de la API.

Finalmente, la correcta implementación de secciones críticas para operaciones globales indispensables (como `transaction_insert`) y el mecanismo de reducción de OpenMP (`reduction(+:...)`) aseguraron la exactitud matemática de las métricas agregadas. El sistema logró consolidar promedios financieros al centavo, procesando el dataset completo sin registrar ninguna consulta fallida no recuperada, probando ser una solución robusta y determinista.

6 Conclusiones

El desarrollo del presente proyecto permitió diseñar e implementar una solución industrial eficiente en C++ para el procesamiento paralelo de transacciones financieras y enriquecimiento demográfico a través de red.

Las conclusiones principales de este estudio son:

- La paralelización basada en directivas de compilador de OpenMP es una herramienta extremadamente potente y de bajo overhead para optimizar aplicaciones híbridas.
- En sistemas dependientes de red (I/O Bound), la concurrencia es el mecanismo principal de optimización, ya que el speedup obtenido está estrechamente ligado al número de canales concurrentes activos y al solapamiento de latencia de red.
- Optimizar a nivel de protocolo de red mediante técnicas como HTTP Keep-Alive y cachear descargas SFTP de archivos locales persistentes representa un impacto de rendimiento equivalente o superior al del paralelismo algorítmico puro, reduciendo el tráfico y la latencia global.
- La robustez en entornos altamente concurrentes exige prever fallos de red físicos y lógicos (como caídas del servidor REST de evaluación o expiraciones de JWT). El diseño de reintentos escalonados en memoria

RAM y el control de exclusión mutua para la renovación de tokens en caliente garantizan la tolerancia a fallos del sistema sin introducir cuellos de botella. Esta resiliencia queda demostrada de forma práctica en el registro de pruebas adjunto en el repositorio (`pruebas_rendimiento.md`), evidenciando la recuperación exitosa ante la expiración en vivo del token sin interrupción del flujo de trabajo.

Referencias

- [1] Mattson, T. G., Sanders, B. A., & Massingill, B. L. (2004). *Patterns for parallel programming*. Addison-Wesley Professional.

7 Anexos

Anexo A: Especificaciones Técnicas

Tabla 4: Configuración del servidor SFTP.

Parámetro	Valor
Protocolo	SFTP
Host	137.184.45.251
Usuario	utem
Contraseña	CPyD.2026
Carpeta inicial	/

Tabla 5: Estructura de los archivos CSV.

Columna	Tipo	Descripción
FECHA	String	Fecha en formato ISO 8601 (UTC-4)
CANAL	String	Canal de compra (POS, online)
SKU	Integer	Identificador único del producto
PRODUCTO	String	Nombre del producto
UNIDADES	Integer	Cantidad de unidades compradas
PORCENTAJE DESC.	Float	Porcentaje de descuento (0 a 1)
MONTO APLICADO	Float	Monto total pagado en CLP
BOLETA	Integer	Número de boleta
LOCAL	String	Identificador del local
CODIGO CLIENTE	String (UUID)	Identificador único del cliente

Tabla 6: Endpoints de la API REST.

Parámetro	Valor
URL base	https://api.sebastian.cl/cpyd
Autenticación	POST /v1/login/authenticate
Consulta persona	GET /v1/person/{uuid}
Cabecera de Auth	Authorization: Bearer <JWT>

Anexo B: Tabla de Dificultades y Soluciones

Tabla 7: Dificultades encontradas y soluciones implementadas.

Dificultad	Solución
1. Saturación y límites de conexión en el servidor SFTP (SSH) El procesamiento paralelo sin control para descargar archivos CSV podía sobrecargar el servidor SFTP o superar los límites de conexiones concurrentes del protocolo SSH.	Se limitó el paralelismo de descarga a un máximo de 4 hilos concurrentes. Cada hilo mantiene activa una única instancia persistente de CURL, lo que permite a <code>libcurl</code> reutilizar la sesión SSH/SFTP subyacente para descargar los archivos secuencialmente sin renegociar la conexión por cada archivo.
2. Expiración de seguridad y concurrencia del Token JWT (HTTP 401) El token JWT requerido por la API expira periódicamente. Si múltiples hilos concurrentes detectan la expiración simultáneamente e intentan renovarlo, se generarían llamadas a la API redundantes y condiciones de carrera.	Se implementó una sección crítica (<code>#pragma omp critical(token_lock)</code>). Cuando un hilo recibe un error HTTP 401, bloquea la sección, renueva el JWT global una única vez, y el resto de los hilos retoma el procesamiento utilizando el token recién actualizado.
3. Inestabilidad de la API REST y gestión de 3 tipos de fallos La API REST presenta intermitencias físicas (timeouts/caídas de red), errores de servidor (HTTP 5xx) y expiraciones (HTTP 401) bajo alta concurrencia, lo que podría comprometer la consistencia de los datos.	Se manejan los 3 fallos (timeouts con <code>api_curl_result != CURLE_OK</code> , caídas de servidor con <code>http_status_code >= 500</code> y expiraciones). Para los primeros dos, los registros se marcan en RAM como "UNKNOWN" y se pasan a una rutina de reintento en caliente que procesa en memoria (mediante vectores) únicamente las peticiones fallidas hasta en 3 pasadas extras con retraso de 2 segundos.
4. Latencia de red crítica en la consulta masiva a la API REST Realizar casi 10,000 consultas HTTP independientes a la API REST generaba un cuello de botella debido al overhead de abrir y cerrar conexiones TCP y negociar handshakes SSL/TLS en cada petición.	Se inicializó el manejador CURL una sola vez por cada uno de los 48 hilos antes de entrar al bucle <code>omp for</code> . Esto forzó el uso automático de HTTP Keep-Alive, manteniendo abiertas las conexiones TCP/TLS activas y reutilizándolas a lo largo de las peticiones asignadas a cada hilo.
5. Desequilibrio de carga (Load Imbalance) por culpa de la latencia de red Las llamadas HTTP a la API REST son altamente impredecibles; algunas responden en milisegundos y otras toman segundos o fallan. Un reparto estático de OpenMP haría que algunos hilos ¹⁶ queden inactivos temprano y otros se saturen, perjudicando el speedup.	Se utilizó la directiva <code>#pragma omp for schedule(dynamic)</code> . Con la planificación dinámica, a los hilos se les asignan tareas a demanda: en cuanto un hilo termina una consulta HTTP, toma de inmediato la siguiente disponible, garantizando el balance de carga y el uso eficiente de los hilos.

Anexo C: Instrucciones de Compilación y Ejecución

Tabla 8: Comandos de compilación y ejecución.

Acción	Comando
Compilación paralela	make
Compilación secuencial	make seq
Ejecución	./farmacia_parallel
Ejecución forzada secuencial	./farmacia_parallel --seq
Limpieza de archivos	make clean
Limpieza total	make distclean

Anexo D: Estructura del Proyecto

```

Paralela_Trabajo_Sem1_UTEM/
|-- main.cpp                # Punto de entrada, orquestación
|-- Makefile                # Sistema de compilación
|-- include/
|   |-- api_manager.hpp     # Declaración consulta API
|   |-- csv_parser.hpp      # Declaración parser CSV
|   |-- metrics_processor.hpp # Declaración procesador métricas
|   |-- sftp_manager.hpp    # Declaración manager SFTP
|   '-- utils.hpp           # Utilidades y callbacks
|-- src/
|   |-- api_manager.cpp     # Implementación consulta API
|   |-- csv_parser.cpp      # Implementación parser CSV
|   |-- metrics_processor.cpp # Implementación métricas
|   |-- sftp_manager.cpp    # Implementación SFTP
|   '-- utils.cpp           # Utilidades
|-- data/                   # Archivos CSV descargados
'-- obj/                    # Archivos objeto compilados

```